TEAM 5 / CEN 4020

EventHub
Implementation & AI Process Report

Part 4 · Call 4 design revision · Python / Flask / SQLite

Jaideep Yarasingu — Gold member | John Bachta — teammate reviewer | October 26, 2026

This is the corrected Gold-member submission package prepared after John's code and documentation review. The revised work lives in a separate Part 4 branch for teammate approval; the shared main branch is not automatically changed by this report.

At a glance

Deliverable	What is included	Grade
1. Design change record	New EARS requirements, supersession links, revised C4 component model, open questions	20%
2. AI development process	Component contracts, concrete review findings, before/after changes, test evidence and prompt improvements	60%
3. Tests	Modular unit tests, concurrency test, Flask integration tests and automated checks	15%
4. Source files	Working prototype, requirement/specification updates, diagram exports and reviewer instructions	5%

Verified status / important limits

This revision: 43 locally passing tests and 2 test modules skipped because this isolated build environment cannot install Flask/Werkzeug. The EARS/C4 eligibility checker reports 0 failures. The previous Mac traceback shows 38 passed / 1 failed before the PBKDF2 compatibility fix; John later cited 49 passed, which I cannot independently reproduce in this environment.

Passing automated checks do not settle missing customer policy emails, discount percentage, audit retention or human approval. The report separates implemented behavior from provisional assumptions and unverified tests.

1. What actually changed after Call 4 (20%)

In Part 3 we had a single modular Flask application, SQLite in WAL mode, pricing in one place and a protected administrator area. For Part 4 we had to make it implementable and independently testable. John correctly pointed out that my first draft had only described those changes in the code while leaving the requirements and C4 model frozen. I have now updated both, with a saved copy of the Call 3 DSL for comparison.

1.1 Requirement changes and preserved history

Call 4 issue	New or revised EARS items	What changed
Uppercase USF email / domain checking	pricing-002 → pricing-011; pricing-012	The discount check is explicitly case-normalized, trimmed and separately testable. Prior IDs and existing identity-003 chain remain linked.
Plus tags and subdomains	pricing-013 and pricing-014	Clearly documented provisional assumption: plus-tags under exact usf.edu eligible; mail.usf.edu subdomain not eligible. Q17 needs customer confirmation.
Audit viewing and retention	audit-005 and audit-006	Administrator-only reading explicitly required. Retention remains [TBD: Q16], not an invented 30/90-day policy.
Standalone tests and shared checks	system-007 and system-008	Service separation and automated unit tests are expressed as new system requirements.

The professor's provided scripts accept the changed requirement files, including the reciprocal supersedes links. All earlier IDs are preserved. The original Part 2-to-3 mapping is still in docs/traceability.md; the added Part 4 change mapping is docs/part4-traceability.md.

1.2 Revised C4 component model

Call 4 specifically asks for discount eligibility as a small, testable function. The new USF Discount Eligibility component is called by Pricing, but it never computes or rounds money. That keeps the earlier one-place pricing rule. The root workspace.dsl and added Part4Components view capture the boundary; the untouched Call 3 source is archived under docs/part4-design-baseline/.

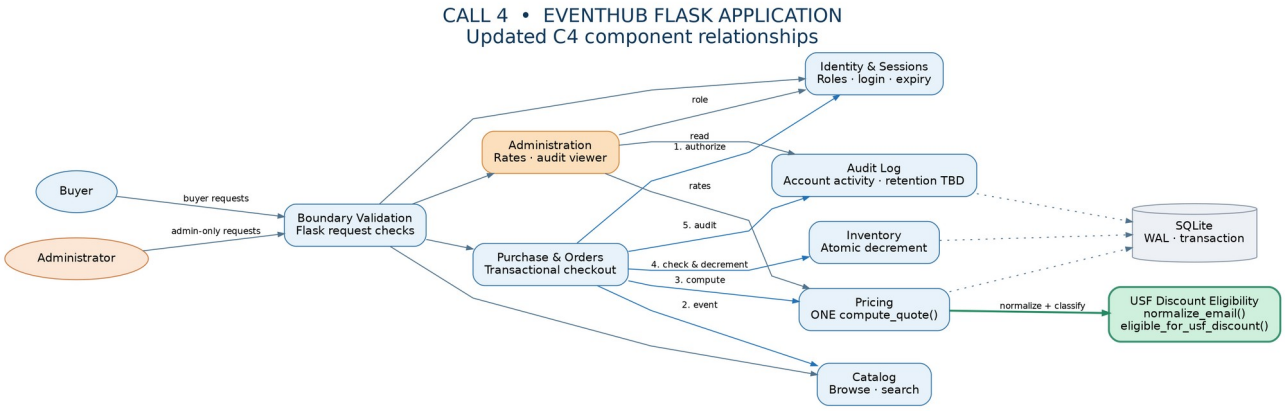


Figure 1. Call 4 component relationships, rendered from the revised architecture in Graphviz; the source of truth is workspace.dsl.

1.3 Open questions were reconciled rather than guessed

Call 4 says customer emails settled ticket holds, discount scope, confirmation and purchase caps. Those email contents were not supplied, so the exact policies cannot be asserted. docs/open-items.md now distinguishes "reported decided, email missing" from genuine TBDs. It also adds Q16 audit retention and Q17 alias/subdomain behavior. The discount percentage is still explicitly unknown in the call, so the prototype blocks a qualifying checkout rather than applying a made-up rate.

2. AI development and correction process (60%)

2.1 How I explained the component diagram to AI

A diagram with boxes and arrows was not enough to generate consistently testable code. The useful format was a small written contract for every component: name, responsibility, entry point, input/output types, invariants, database tables, collaborators, failure behavior and a sample test. This gave AI concrete instructions and gave us something to compare its answer against during code review.

Contract	Concrete specification used for the final build
Eligibility	normalize_email(value) → normalized string; eligible_for_usf_discount(email) → boolean; casefold and exact @usf.edu domain; aliases allowed and subdomains excluded provisionally. No SQL or price arithmetic.
Pricing	compute_quote(base_usd_minor:int, quantity:int, eligible:bool, currency:str, decimal rate, optional discount) → Quote with whole minor units. Money computed once using Decimal; JPY has zero decimals; missing approved discount is an error.
Inventory / Purchase	BEGIN IMMEDIATE; check and decrement as a single SQL step; do not oversell; commit order, stock, and success audit together; failed attempt rolls back and is then audited.
Administration / Audit	Authenticated admin role is verified on the server; rates are validated and audited; audit viewer restricted to admins; retention pending policy decision.

This contract approach also explains why the final project has separate eventhub modules and dedicated tests. I now treat the C4 view as a map of logical responsibilities and use the contract cards to specify the interfaces that generated code must satisfy.

2.2 How instructions changed during the work

The lines below are concise reconstructed prompt examples based on the actual changes, not fake word-for-word transcripts. The original drafting log only preserved prompt summaries. I'm including them because the assignment asks what kind of instruction produced better code.

Less effective initial direction	More testable corrective direction
"Build Flask login and store hashed passwords."	"Specify the Werkzeug hashing method explicitly as PBKDF2-SHA256 with 600,000 iterations; run the existing authentication regression tests on the target Mac."
"Keep Part 3 intact while building Call 4."	"Keep IDs and original diagrams for history, but supersede changed EARS requirements and add a separate eligibility component/edge to the Call 4 model. Run the supplied eligibility checker."
"Make pricing support USF discounts."	"Create one small normalize-and-classify function with uppercase, plus-tag, exact-domain and subdomain examples. Pricing calls the Boolean result but remains the only money calculator."

2.3 Was the generated code correct the first time?

No. It had a working skeleton and useful domain tests, but there were correctness and review gaps. One failure was observed when running the full dependencies on a Mac. Other gaps were spotted by human reading rather than by pytest: the old diagram missed the eligibility boundary, the requirements never mentioned email normalization, the report gave an outdated test count, and the README implied the wrong version was current. That is why I do not treat the first green badge as final acceptance.

Evidence A — real Mac error and exact code change

```
Before: generate_password_hash(password)
Mac traceback: AttributeError: module 'hashlib' has no attribute 'scrypt'
Observed Mac summary: 1 failed, 38 passed
After: generate_password_hash(password, method='pbkdf2:sha256:600000')
```

The earlier code relied on Werkzeug choosing a default hashing method. On that Mac/Python combination, scrypt was unavailable. I changed the method explicitly to PBKDF2-SHA256 and retained the authentication test in tests/test_identity.py. John later reported a 49-passing-test run, but because that output is not attached here, I label it as teammate-reported evidence rather than an independently repeated count.

Evidence B — requirement and model repair after John's review

```
Part 3: pricing-002  ->  [marked superseded by pricing-011]
Part 4: pricing-011  ->  supersedes pricing-002
Added:  pricing-012 / pricing-013 / pricing-014
New C4: Pricing -> USF Discount Eligibility
New code: eventhub/eligibility.py; standalone tests/test_eligibility.py
```

This correction fixes the actual specification/model discrepancy that John found. I did not just write a new paragraph claiming the software normalizes emails. The requirement, model, source code and tests now all express the same boundary, while unapproved alias/subdomain rules are clearly labeled assumptions.

Evidence C — reviewer-requested privacy and quality changes

- Added audit-005 for administrator-only reading and audit-006/Q16 to acknowledge that retention policy remains unknown. The audit service has its own independent test file.
- Added Ruff format checking in the workflow and optional local hook; supplied a fix-and-verify script because Ruff is not available in this sandbox. The latest commit must pass those checks before merging.
- Expanded CONTRIBUTING.md with the four required completion boxes and clarified that branch protection is a separate maintainer-controlled GitHub setting.
- Added docs/review-response.md to respond to each of John's review comments rather than leaving follow-up work buried in Discord.

2.4 Human review mattered

John reviewed the report as a teammate and highlighted issues that automated tests did not catch: requirements were frozen despite the changed call, no dedicated C4 eligibility component existed, the process report did not explain how the diagram became prompts, test facts were stale and several policy statements contradicted each other. I used his findings as a concrete checklist and updated the artifacts that caused the mismatch. John still needs to approve the final revision, especially the policy questions that depend on missing emails.

2.5 What I would change in the next AI coding cycle

I would not ask the AI for the whole application at once. I would supply the latest approved EARS requirements, the diagram and contract for one component, and three or four acceptance tests. I would ask for that module and its tests together, review the actual diff, run those tests, and then integrate it. Any correction request would include the failing assertion or traceback and tell the AI which invariant cannot change. This prevents vague “fix everything” responses and leaves a clearer review trail.

3. Unit tests, CI and proof (15%)

3.1 Results obtained for this revision

Evidence	Result	What it means
Local pytest (this build environment)	43 passed, 2 skipped	Domain tests and newly separated eligibility, inventory and audit tests pass. Flask/Werkzeug could not be installed, so identity and HTTP modules were not run here.
Part 3 eligibility scripts	0 failures	EARS syntax, supersession links, C4 declaration/view files and directory rules are structurally valid. Structurizr CLI validation is unavailable here.
Python compilation	Passed	All application and test modules parse.
Earlier Mac test run (screenshot)	38 passed, 1 failed	Real default-scrypt compatibility bug; fixed to PBKDF2.
Teammate result	49 passed (reported)	John’s statement; not re-run in this build environment.
GitHub Actions at older PR commit	2 successful checks (user screenshot)	Earlier branch validation only; the latest review revision must have fresh green checks.
Ruff full formatting	Not verified locally	Formatter unavailable in sandbox; upload script runs Ruff format and CI runs a blocking format check.

The scripts and tests are included so John can rerun the missing environment-dependent checks. This report deliberately does not describe skipped tests as passing, or the current review revision as already approved or merged.

3.2 Highest-risk test cases

- **Oversell:** two buyers attempt to purchase the final ticket through separate SQLite connections; one succeeds, one fails, the count ends at zero and two audit attempts exist.
- **Money:** USD cents, conversion to EUR, Japanese yen with no fractional unit, missing student-discount configuration and invalid exchange rates.
- **Eligibility:** mixed-case USF emails, whitespace, plus aliases, subdomains, lookalike domains and missing addresses are independently tested.
- **Audit / inventory:** separate module-level tests cover newest-first audit reads, record detail limits, atomic decrement and sold-out rejection.
- **HTTP / access:** supplied Flask tests cover CSRF, session expiry, safe redirects, role-based admin denial, login, quote validation and browser workflows; these must pass on the teammate’s dependency-ready machine and the new PR commit.

3.3 Commands needed before review approval

```
python -m pip install -r requirements-dev.txt
```

```
ruff format eventhub tests scripts
ruff format --check eventhub tests scripts
ruff check eventhub tests scripts --select E4,E7,E9,F63,F7,F82
python -m compileall -q eventhub tests scripts
python -m pytest -q
python grading/check_part3.py .
```

After pushing the updated review branch, check the fresh GitHub Actions run for a full zero-skip test result. Only then is the automatic gate demonstrated. The repository maintainer must still require the check before merge and review the changed code.

4. Final code and teammate handoff (5%)

The ZIP contains the modular working prototype, all existing Part 3 documentation, new Part 4 EARS and C4 changes, all unit tests, GitHub Actions configuration, reports, and a reviewer checklist. There is no real payment gateway: checkout is explicitly simulated. Example events and rates are demo data.

Location	What John should check
eventhub/; run.py; templates/; static/	Flask app behavior, modular service interfaces, browser flows, secure roles and integer-money calculations.
features/; docs/part4-traceability.md	New EARS IDs and reciprocal supersession links; customer decisions not silently invented.
workspace.dsl; diagrams/Part4Components.*	Pricing → independently testable Discount Eligibility and original context/transaction boundaries.
tests/; report/local_test_results.txt	Independent component tests, 2-writer race, environment-dependent HTTP tests and current result limits.
CONTRIBUTING.md; .github/workflows/quality.yml	Four-box completion rules, Ruff format/lint, CI, and need for branch protection.
docs/review-response.md; docs/open-items.md	John's exact feedback addressed; Q1/Q2/Q4/Q5/Q6/Q16/Q17 remaining source-of-truth needs.

Before we submit

- John reviews the change diff, especially revised requirements and the independent Eligibility component; any changes must be reflected in the report.
- Obtain the customer emails for discount scope, ticket holds, confirmation and purchase caps; apply their actual rules. Do not invent the discount percentage or the audit-retention period.
- Run the format fixer and all tests in a dependency-ready environment, push the updated review branch, confirm fresh CI checks and a reviewer approval; enable required checks on main.
- Submit the finished report and the source/test files as required by Canvas; the Part 4 PDF did not specify a separate reflection or a fixed ZIP filename.

I made the code and documentation corrections in response to John's review so that the written requirements, component model and software describe the same design. The implementation is ready for his next review, but the customer-policy and full environment checks remain genuine release blockers.